

```

# =====
# LOGISTIC REGRESSION CAPSTONE
# PREDICTING USER VERIFICATION STATUS
# COMPLETE SUBMISSION
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)
from sklearn.utils import resample

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

print("="*60)
print("LOGISTIC REGRESSION CAPSTONE")
print("PREDICTING USER VERIFICATION STATUS")
print("="*60)

# =====
# FIND CSV FILE
# =====

print("\n" + "="*60)
print("FINDING CSV FILE...")
print("="*60)

csv_files = [f for f in os.listdir('.') if f.endswith('.csv')]

if csv_files:
    filename = csv_files[0]
    print(f"Found: {filename}")
    df = pd.read_csv(filename)
else:
    print("ERROR: CSV file not found. Please upload the CSV file.")
    print("Run: from google.colab import files; uploaded = files.upload()")
    exit()

print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")

# =====
# MILESTONE 1: DISCOVERY & PREPROCESSING
# =====

print("\n" + "="*60)
print("MILESTONE 1: DISCOVERY & PREPROCESSING")
print("="*60)

```

```

print("\nFirst 5 rows:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

# Handle missing values
df = df.dropna()
print(f"\nAfter dropping missing values: {df.shape[0]} rows")

# Identify target column (verification status)
target_col = None
for col in df.columns:
    if 'verified' in col.lower() or 'status' in col.lower() or 'target' in col
        target_col = col
        break

if target_col is None:
    # Assume last column is target
    target_col = df.columns[-1]

print(f"Target column: {target_col}")

X = df.drop(target_col, axis=1)
y = df[target_col]

print(f"\nTarget distribution before balancing:")
print(y.value_counts())

# =====
# CREATE TEXT_LENGTH FEATURE
# =====

print("\n" + "="*60)
print("CREATE TEXT_LENGTH FEATURE")
print("="*60)

# Find text columns (object/string columns)
text_cols = X.select_dtypes(include=['object']).columns

if len(text_cols) > 0:
    # Create text_length feature from first text column
    first_text_col = text_cols[0]
    X['text_length'] = X[first_text_col].astype(str).str.len()
    print(f"Created 'text_length' from column: {first_text_col}")
else:
    # If no text columns, create from a numeric column
    numeric_cols = X.select_dtypes(include=['number']).columns
    if len(numeric_cols) > 0:
        X['text_length'] = np.abs(X[numeric_cols[0]]) # Use absolute value
        print(f"Created 'text_length' from: {numeric_cols[0]}")
    else:
        X['text_length'] = 0
        print("No suitable column for text_length. Set to 0.")

```

```

print(f"Features now include: {X.columns.tolist()}")

# =====
# HANDLE CLASS IMBALANCE (UPSAMPLING)
# =====

print("\n" + "="*60)
print("HANDLE CLASS IMBALANCE - UPSAMPLING")
print("="*60)

# Combine X and y for resampling
data = pd.concat([X, y], axis=1)

# Separate majority and minority classes
majority = data[data[target_col] == data[target_col].value_counts().idxmax()]
minority = data[data[target_col] == data[target_col].value_counts().idxmin()]

print(f"Majority class: {len(majority)} samples")
print(f"Minority class: {len(minority)} samples")

# Upsample minority class
minority_upsampled = resample(minority,
                              replace=True,
                              n_samples=len(majority),
                              random_state=42)

# Combine majority and upsampled minority
data_balanced = pd.concat([majority, minority_upsampled])

# Shuffle
data_balanced = data_balanced.sample(frac=1, random_state=42).reset_index(drop=True)

print(f"Balanced dataset: {data_balanced.shape[0]} samples")

# Separate X and y again
X_balanced = data_balanced.drop(target_col, axis=1)
y_balanced = data_balanced[target_col]

print(f"\nTarget distribution after balancing:")
print(y_balanced.value_counts())

# =====
# MILESTONE 2: FEATURE ENGINEERING & ENCODING
# =====

print("\n" + "="*60)
print("MILESTONE 2: FEATURE ENGINEERING & ENCODING")
print("="*60)

# Encode categorical variables
categorical_cols = X_balanced.select_dtypes(include=['object']).columns

if len(categorical_cols) > 0:
    print(f"Categorical columns: {categorical_cols.tolist()}")
    X_encoded = pd.get_dummies(X_balanced, columns=categorical_cols, drop_first=True)
    print("One-Hot Encoding applied.")
else:
    X_encoded = X_balanced.copy()
    print("No categorical columns to encode.")

```

```

print(f"Encoded features shape: {X_encoded.shape}")

# =====
# CORRELATION HEATMAP
# =====

print("\n" + "="*60)
print("CORRELATION HEATMAP")
print("="*60)

# Select only numeric columns
numeric_data = X_encoded.select_dtypes(include=['number'])

# Calculate correlation matrix
corr_matrix = numeric_data.corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', center=0)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()

# Identify highly correlated features (correlation > 0.9)
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr = [(column, row) for column in upper_tri.columns for row in upper_tri.index if column < row]

print(f"\nHighly correlated feature pairs (correlation > 0.9):")
if high_corr:
    for pair in high_corr:
        print(f" {pair[0]} and {pair[1]}")
else:
    print(" No highly correlated pairs found.")

# Drop one feature from highly correlated pairs
drop_cols = set()
for col, row in high_corr:
    if col in X_encoded.columns:
        drop_cols.add(col)

if drop_cols:
    X_encoded = X_encoded.drop(columns=drop_cols)
    print(f"Dropped {len(drop_cols)} highly correlated features.")

print(f"Features shape after dropping multicollinear features: {X_encoded.shape}")

# =====
# MILESTONE 3: MODEL CONSTRUCTION
# =====

print("\n" + "="*60)
print("MILESTONE 3: MODEL CONSTRUCTION")
print("="*60)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y_balanced, test_size=0.2, random_state=42)

print(f"Training set: {X_train.shape[0]} rows")
print(f"Testing set: {X_test.shape[0]} rows")

```

```

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build Logistic Regression model
model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
y_proba = model.predict_proba(X_test_scaled)[: , 1]

print("Model trained successfully!")

# =====
# MILESTONE 4: PERFORMANCE EVALUATION
# =====

print("\n" + "="*60)
print("MILESTONE 4: PERFORMANCE EVALUATION")
print("="*60)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_proba)

print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-Score:   {f1:.4f}")
print(f"ROC-AUC:    {roc_auc:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Logistic Regression')
plt.legend()
plt.show()

```

```

# =====
# MILESTONE 5: ACTIONABLE INSIGHTS
# =====

print("\n" + "="*60)
print("MILESTONE 5: ACTIONABLE INSIGHTS")
print("="*60)

# Get coefficients
coef_df = pd.DataFrame({
    'Feature': X_encoded.columns,
    'Coefficient': model.coef_[0]
}).sort_values('Coefficient', ascending=False)

print("\nTop features driving verification (positive coefficients):")
print(coef_df.head(10))

print("\nTop features driving non-verification (negative coefficients):")
print(coef_df.tail(10))

print("\n" + "="*60)
print("BUSINESS RECOMMENDATIONS")
print("="*60)

print(f"""
💡 MODEL PERFORMANCE SUMMARY:
• Accuracy: {accuracy*100:.1f}%
• Precision: {precision*100:.1f}%
• Recall: {recall*100:.1f}%
• F1-Score: {f1*100:.1f}%
• ROC-AUC: {roc_auc*100:.1f}%

✅ BUSINESS RECOMMENDATIONS:

1. FOCUS ON TOP FEATURES:
  - The following features most strongly predict verification:
    {coef_df.head(3)['Feature'].tolist()}

2. ADDRESS NEGATIVE FEATURES:
  - Features with negative coefficients reduce verification likelihood

3. DATA BALANCING IS CRITICAL:
  - Class imbalance was addressed using upsampling
  - This improves model performance on minority class

4. TEXT_LENGTH FEATURE:
  - Created 'text_length' to capture textual information
  - This may help predict verification status

5. MONITOR AND RETRAIN:
  - Model should be retrained periodically
  - Track F1-score and ROC-AUC over time

6. FEATURE ENGINEERING:
  - Added 'text_length' for better predictions
  - One-Hot Encoding for categorical variables
  - Removed multicollinear features
""")

```

```
print("="*60)
print("PROJECT COMPLETE - ALL MILESTONES MET")
print("="*60)
```